

ROSIT+Ubuntu 基础

对，这两个地方应该改，而且会明显提升教材的“入口友好度”。

我建议这样处理：

第一点：Linux 基础不要只放附录。

原来的“附录 A Linux 命令基础”太靠后了，学生真正卡住的地方往往一开始就出现：`cd`、`source`、`chmod +x`、`catkin_make`、`.bashrc`、多个终端、路径、权限、环境变量。

所以应该把 Linux 基础前置成**“第零篇”**，但不要写成传统 Linux 教程，而是写成：

为了运行 ROS，我们必须先学会怎样和 Linux 实验环境打交道。

第二点：roscore 不能直接从天而降。

应该先铺垫一个问题：

如果机器人系统里有很多程序，它们怎么知道彼此在哪里？

谁负责帮它们登记？

谁让控制程序找到仿真程序？

谁让键盘节点把速度命令送给小乌龟？

这样再引出 `roscore`，学生会觉得它是“自然出现的工具”，而不是“第一条必须背的命令”。

下面是我建议你替换到正式目录里的**新版开篇结构**。

《从小乌龟到机器人系统：ROS1 实践入门》

修订版目录结构：增加 Linux 前导篇，并重新设计 ROS 入门顺序

前言

为什么这本书从 Linux 开始，而不是直接从 ROS 开始

学习 ROS 的困难，往往不完全来自 ROS 本身。

很多初学者还没有真正理解：

- 终端是什么；
- 路径是什么；
- 程序为什么要有执行权限；
- 为什么每开一个新终端都要重新 source；
- 为什么一个 ROS 系统需要同时开好几个程序；
- 为什么一个命令在这个终端能运行，换个终端就找不到；
- 为什么 Python 脚本写好了却无法执行；
- 为什么编译之后还要 source 工作空间。

如果这些基础没有铺好，学生会把 Linux 问题、环境问题、ROS 问题混在一起。

他们会以为自己“ROS 没学会”，其实只是“不知道程序在哪里”“不知道环境变量是什么”“不知道终端之间互不共享状态”。

所以本书不直接从 `roscore` 开始。

我们先从一个更朴素的问题开始：

一台机器人通常不是由一个程序控制的。

那么，多个程序怎样同时运行？

它们怎样找到彼此？

它们怎样交换数据？

它们怎样被组织成一个完整系统？

当这个问题出现之后，ROS 才有意义。

当 ROS 有意义之后，`roscore` 才不会显得突兀。

当 `roscore` 被理解为“节点登记和通信发现的中心”之后，小乌龟的登场才会自然。

第零篇 进入机器人实验室：ROS 需要的 Linux 基础

第 0 章 开始之前：我们到底要学什么

0.1 ROS 不是一个单独的软件

ROS 更像是一套机器人软件的组织方法。

它帮助很多程序同时工作。

0.2 一台机器人为什么需要很多程序

以一个简单移动机器人为例，它可能同时需要：

- 读取键盘或手柄输入；
- 接收传感器数据；
- 估计当前位置；
- 规划运动路径；
- 控制电机速度；
- 显示运行状态；
- 记录实验数据。

这些功能不适合全部写在一个巨大程序里。

0.3 本书的学习路线

本书从三个层次逐步展开：

Plain Text

1 Linux 基础 -> 多程序协作 -> ROS 通信系统 -> turtlesim 小乌龟实验 -> 完整机器人应用

0.4 本书为什么选择 turtlesim

turtlesim 足够小，但能承载 ROS 的核心概念：

- node；
- topic；
- message；
- service；
- parameter；

- launch;
- namespace;
- action;
- tf 思想;
- rosbag;
- 多机器人协同;
- 状态机;
- 任务系统。

它是一个二维机器人实验场。

第 1 章 终端：和机器人实验环境对话

1.1 为什么 ROS 学习离不开终端

ROS 的很多操作都发生在终端中：

- 启动节点;
- 查看话题;
- 调用服务;
- 编译工作空间;
- 记录数据;
- 运行脚本;
- 查看日志。

所以，终端不是“额外知识”，而是 ROS 实验的基本操作界面。

1.2 当前目录：程序从哪里开始找文件

核心命令：

Plain Text

```
1 pwd
2 ls
3 cd
```

讲清楚：

- `pwd`：我现在在哪里；
- `ls`：这里有什么；
- `cd`：我要去哪里。

1.3 绝对路径与相对路径

举例：

Plain Text

```
1 /home/student/catkin_ws/src
2 ./scripts/talker.py
3 ../config/params.yaml
```

引导学生理解：

很多 ROS 报错，本质上是程序没有在正确的位置找到文件。

1.4 创建、复制、删除文件和目录

核心命令：

Plain Text

```
1 mkdir
2 touch
3 cp
4 mv
5 rm
```

1.5 本章练习

1. 创建一个名为 `ros_practice` 的目录。
2. 在其中创建 `scripts`、`config`、`launch` 三个子目录。
3. 使用 `pwd` 查看当前位置。
4. 使用相对路径进入上一级目录。
5. 删除一个测试文件。

1.6 本章小结

本章只要求学生记住一句话：

ROS 工程首先是 Linux 文件系统中的工程。
不知道自己在哪个目录，就很难知道 ROS 为什么找不到文件。

第 2 章 文件、脚本与权限：为什么程序写好了却不能运行

2.1 文件不等于程序

一个 Python 文件写好了，并不代表它一定能被系统当作程序执行。

2.2 查看文件权限

核心命令：

Plain Text

```
1 ls -l
```

解释：

Plain Text

```
1 -rwxr-xr-x
```

重点讲：

- r: 可读；
- w: 可写；
- x: 可执行。

2.3 给 Python 脚本添加执行权限

核心命令：

Plain Text

```
1 chmod +x scripts/hello.py
```

2.4 shebang: 告诉系统用谁运行脚本

Python 脚本开头：

Plain Text

```
1 #!/usr/bin/env python3
```

解释：

这行不是注释，而是在告诉 Linux：请用 python3 来运行这个文件。

2.5 为什么 rosrun 找不到我的脚本

常见原因：

- 脚本没有执行权限；
- 脚本不在 package 的正确目录；
- 文件名写错；
- 没有 source 工作空间；
- package 没有被正确编译或识别。

2.6 本章练习

1. 创建一个 Python 脚本。
 2. 不加执行权限直接运行，观察错误。
 3. 使用 `chmod +x` 添加执行权限。
 4. 加入 shebang 后再次运行。
 5. 故意写错文件名，观察终端提示。
-

第 3 章 软件安装、依赖与版本：ROS 为什么这么重视环境

3.1 为什么同一份代码在别人电脑上跑不起来

常见原因：

- ROS 版本不同；
- Python 版本不同；
- 缺少依赖包；
- 工作空间没有编译；
- 环境变量没有配置；
- Ubuntu 版本不匹配。

3.2 apt：安装系统软件包

核心命令：

Plain Text

```
1 sudo apt update
2 sudo apt install package_name
```

3.3 安装 ROS 包

示例：

Plain Text

```
1 sudo apt install ros-noetic-turtlesim
```

讲清楚 ROS 包命名规律：

Plain Text

```
1 ros-发行版名称-包名
2 ros-noetic-turtlesim
3 ros-noetic-geometry-msgs
4 ros-noetic-rqt-graph
```

3.4 git：获取课程代码

核心命令：

Plain Text

```
1 git clone
2 git status
3 git pull
```

3.5 版本固定的重要性

建议课程统一：

- Ubuntu 20.04；
- ROS Noetic；
- Python 3；
- 统一虚拟机或 Docker 镜像；
- 统一课程代码仓库。

3.6 本章练习

1. 使用 apt 安装 turtlesim。

2. 查询一个 ROS 包是否已经安装。
 3. 使用 git 下载课程代码。
 4. 查看当前 Ubuntu 版本。
 5. 查看当前 ROS 发行版环境变量。
-

第 4 章 环境变量与 source：为什么换个终端命令就失效了

4.1 一个常见现象

学生经常遇到：

Plain Text

```
1 rosrun: command not found
```

或者：

Plain Text

```
1 package 'xxx' not found
```

这不一定是 ROS 安装坏了。

很多时候只是当前终端“不知道 ROS 在哪里”。

4.2 什么是环境变量

用直观方式解释：

环境变量就像一张当前终端手里的地图。

地图上写着：命令去哪里找，包去哪里找，库去哪里找。

4.3 查看环境变量

核心命令：

Plain Text

```
1 echo $PATH
2 echo $ROS_PACKAGE_PATH
3 echo $ROS_DISTRO
```

4.4 source 的作用

核心命令：

Plain Text

```
1 source /opt/ros/noetic/setup.bash
```

解释：

source 不是启动 ROS。
source 是把 ROS 的路径信息加载到当前终端。

4.5 工作空间里的 source

当我们创建自己的 catkin 工作空间后，还需要：

Plain Text

```
1 source ~/catkin_ws/devel/setup.bash
```

解释：

这一步告诉当前终端：除了系统 ROS 包，还要认识我们自己写的 ROS 包。

4.6 .bashrc：让每个新终端自动 source

示例：

Plain Text

```
1 echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
2 echo "source ~/catkin_ws/devel/setup.bash" >> ~/.bashrc
```

提醒：

初学阶段不要随便往 `.bashrc` 里加很多行。
每加一行，都要知道它的作用。

4.7 本章练习

1. 打开新终端，查看 `ROS_DISTRO`。
 2. 手动 source ROS 环境。
 3. 创建一个临时工作空间并 source。
 4. 修改 `.bashrc`，再打开新终端验证。
 5. 故意不 source 工作空间，观察 package not found 错误。
-

第 5 章 进程与多终端：机器人系统为什么要同时运行很多程序

5.1 一个程序做所有事情会怎样

如果一个程序同时负责：

- 读取键盘；
- 控制速度；
- 显示位置；
- 记录数据；
- 处理任务；

它会越来越难维护。

5.2 什么是进程

用简单语言解释：

一个正在运行的程序，就是一个进程。

5.3 多个终端就是多个实验窗口

在 ROS 学习中，常常需要多个终端：

- 一个终端启动核心服务；
- 一个终端启动仿真；
- 一个终端启动控制程序；
- 一个终端查看话题；
- 一个终端记录数据。

5.4 查看正在运行的程序

核心命令：

Plain Text

```
1 ps
2 top
3 htop
```

5.5 结束程序

核心操作：

Plain Text

```
1 Ctrl + C
```

以及：

Plain Text

```
1 kill
```

5.6 本章练习

1. 同时打开三个终端。
 2. 在不同终端运行不同 Python 程序。
 3. 使用 `ps` 查看进程。
 4. 使用 `Ctrl + C` 停止程序。
 5. 思考：为什么机器人系统适合拆成多个进程？
-

第一篇 小乌龟出场前：从多程序协作到 ROS

第 6 章 先不讲 ROS：两个程序怎样合作

6.1 一个小问题

现在我们有两个普通程序：

- 程序 A：负责产生速度命令；
- 程序 B：负责接收速度命令并执行。

问题是：

程序 A 怎样把数据交给程序 B？

6.2 最笨的方法：写入文件

程序 A 把速度写进文件。

程序 B 不断读取文件。

优点：

- 容易理解；
- 可以保存数据。

缺点：

- 延迟大；
- 不适合高频控制；

- 多个程序同时读写容易混乱。

6.3 稍好一点的方法：网络通信

程序 A 通过网络发送数据。

程序 B 通过网络接收数据。

优点：

- 可以跨电脑；
- 更接近真实机器人系统。

缺点：

- 每个程序都要处理连接、地址、端口、协议；
- 系统一复杂，管理成本很高。

6.4 真正的问题：不是不能通信，而是通信太难管理

机器人系统里可能有很多程序：

Plain Text

- 1 键盘控制程序
- 2 底盘控制程序
- 3 传感器程序
- 4 定位程序
- 5 地图程序
- 6 路径规划程序
- 7 数据显示程序
- 8 数据记录程序

它们之间需要不断交换数据。

如果每两个程序都自己建立通信连接，系统会非常混乱。

6.5 我们需要一个机器人通信框架

这个框架至少应该解决几个问题：

- 每个程序有名字；

- 程序启动后可以被别人找到；
- 数据有统一格式；
- 程序之间可以一对一通信；
- 程序之间可以一对多通信；
- 可以查看系统里有哪些程序；
- 可以查看系统里有哪些数据正在流动。

6.6 本章小结

本章真正想引出的不是某个命令，而是一个需求：

当机器人系统由多个程序组成时，我们需要一套机制，让这些程序能够发现彼此、交换数据、被调试、被组织。

这个机制，就是 ROS 要解决的问题。

第 7 章 ROS 的出现：给机器人程序一个共同世界

7.1 ROS 解决的不是“写程序”，而是“组织程序”

ROS 不负责替我们写控制算法。

它负责帮助我们组织机器人系统里的多个程序。

7.2 ROS 中的程序叫节点

一个节点可以负责一件事：

- 键盘输入节点；
- 速度控制节点；
- 位置显示节点；
- 轨迹记录节点；
- 任务管理节点。

7.3 节点之间通过通信连接

ROS 提供多种通信方式：

Plain Text

- | | |
|----------|--------------|
| 1 连续数据流 | -> Topic |
| 2 一次请求响应 | -> Service |
| 3 长时间任务 | -> Action |
| 4 参数配置 | -> Parameter |
| 5 坐标关系 | -> TF |
| 6 实验记录 | -> rosbag |

这里先不急着想全部讲。

我们只需要知道：ROS 是为了让多个节点合作。

7.4 那么，节点怎样找到彼此

这时再引出 roscore。

解释：

在 ROS1 中，节点启动后需要一个“登记处”。
节点告诉登记处：我是谁，我发布什么数据，我订阅什么数据。
其他节点也来登记。
这样，节点之间才能找到彼此。

这个登记处，就是 `roscore` 提供的核心功能。

7.5 roscore 不神秘

`roscore` 可以先被理解成三件事的组合：

- 节点登记处；
- 参数服务器；
- 日志相关服务。

初学阶段只需要记住：

没有 roscore，ROS1 节点通常无法正常发现彼此。

7.6 本章小结

现在 `roscore` 不是一个突兀命令，而是自然答案：

因为机器人系统里有很多节点，所以节点需要一个共同世界。
在 ROS1 中，`roscore` 就是这个共同世界的入口。

第二篇 小乌龟醒来：第一个 ROS 实验世界

第 8 章 第一次启动 ROS 世界

8.1 现在再启动 roscore

核心命令：

Plain Text

```
1 roscore
```

这一次，学生已经知道它的意义：

我们不是在背命令，而是在启动 ROS1 节点的登记处。

8.2 打开第二个终端

提醒：

每一个新终端都要确认环境是否已经 source。

检查：

Plain Text

```
1 echo $ROS_DISTRO
```

8.3 启动小乌龟

核心命令：

Plain Text

```
1 rosrun turtlesim turtlesim_node
```

观察：

- 屏幕上出现一个二维窗口；
- 小乌龟处于某个初始位置；
- 程序正在运行，没有立刻退出。

8.4 这时系统里有什么

查看节点：

Plain Text

```
1 rosnode list
```

查看话题：

Plain Text

```
1 rostopic list
```

这一步非常关键。

不要急着控制乌龟，先让学生看到：

一个图形窗口背后，其实已经有了 ROS 节点和 ROS 话题。

8.5 本章练习

1. 启动 `roscore`。
2. 启动 `turtlesim_node`。
3. 使用 `rosnode list` 查看节点。
4. 使用 `rostopic list` 查看话题。

5. 关闭 turtlesim，再观察节点列表变化。

第 9 章 键盘为什么能控制小乌龟

9.1 启动键盘控制节点

核心命令：

Plain Text

```
1 rosrun turtlesim turtle_teleop_key
```

观察：

- 键盘终端获得焦点时，小乌龟可以运动；
- 鼠标点到其他窗口时，键盘可能失效；
- 小乌龟运动后会留下轨迹。

9.2 画出最小通信图

Plain Text

```
1 turtle_teleop_key ---> /turtle1/cmd_vel ---> turtlesim_node
```

解释：

- 键盘节点负责发布速度；
- turtlesim 节点负责接收速度；
- 中间的数据通道叫 topic；
- 速度消息的类型是 `geometry_msgs/Twist`。

9.3 用 rostopic echo 看见速度命令

核心命令：

Plain Text

```
1 rostopic echo /turtle1/cmd_vel
```

让学生看到：

- 按上键时 linear.x 改变；
- 按左右键时 angular.z 改变；
- 不按键时消息可能停止。

9.4 用 rqt_graph 看见节点连接

核心命令：

Plain Text

```
1 rqt_graph
```

观察：

- 哪个节点发布；
- 哪个节点订阅；
- 数据从哪里流向哪里。

9.5 本章小结

这一章要让学生形成第一个 ROS 图像：

ROS 不是一个程序控制另一个程序，
而是一个节点把消息发布到话题上，另一个节点从话题中接收消息。

第 10 章 不用键盘：写程序控制小乌龟

10.1 从观察走向控制

前面我们已经看到键盘节点会发布 `/turtle1/cmd_vel`。

现在我们自己写一个节点，代替键盘节点。

10.2 第一个速度发布节点

任务：

让小乌龟持续向前运动。

10.3 Twist 消息

解释：

Plain Text

```
1 linear.x    前进/后退速度
2 angular.z   左右旋转速度
```

10.4 画圆

任务：

同时给 linear.x 和 angular.z 赋值，让小乌龟画圆。

10.5 撞墙现象

观察：

- 小乌龟不知道墙在哪里；
- 控制节点只知道发速度；
- 控制节点还没有读取位置反馈。

自然引出下一章：

想让小乌龟不撞墙，就必须知道它在哪里。

后续篇章保持原主线，但顺序略作调整

从这里开始，可以接回原来的 turtlesim 主线：

第 11 章 位置反馈：订阅 `/turtle1/pose`

核心内容：

- subscriber;
- callback;
- turtlesim/Pose;
- rqt_plot;
- 判断是否靠近边界。

第 12 章 闭环控制：让小乌龟自动走向目标点

核心内容：

- 目标点;
- 距离误差;
- 角度误差;
- 比例控制;
- 到达判定。

第 13 章 Message：通信数据不是随便写的

核心内容：

- rosmmsg;
- geometry_msgs/Twist;
- turtlesim/Pose;
- 自定义 TurtleStatus.msg。

第 14 章 Service：让小乌龟接受一次性请求

核心内容：

- /spawn;
- /clear;
- /kill;
- /teleport_absolute;

- 自定义服务。

第 15 章 Action：让小乌龟执行可取消的长任务

核心内容：

- goal;
- feedback;
- result;
- cancel;
- 多目标任务。

第 16 章 Parameter：给小乌龟设置性格

核心内容：

- rosparam;
- 私有参数;
- YAML;
- 不同乌龟不同参数。

第 17 章 Package：把零散程序整理成项目

核心内容：

- catkin_create_pkg;
- package.xml;
- CMakeLists.txt;
- scripts;
- msg/srv/action/config/launch。

第 18 章 Launch：一键启动小乌龟系统

核心内容：

- node;
- param;
- rosparam;

- remap;
- include;
- namespace。

第 19 章 Debug：小乌龟不动时怎么办

核心内容：

- rosnodet;
- rostopic;
- rosservice;
- rosparm;
- rqt_graph;
- rqt_plot;
- 固定排错流程。

后续多乌龟、追踪、编队、TF、rosbag、状态机、综合项目继续保留。

我建议最终结构采用这个节奏：

Plain Text

- 1 第零篇：Linux 与多程序基础
- 2 第一篇：ROS 出现的理由
- 3 第二篇：小乌龟正式出场
- 4 第三篇：Topic 与 Message
- 5 第四篇：Service、Action、Parameter
- 6 第五篇：Package、Launch、Debug
- 7 第六篇：多乌龟协同
- 8 第七篇：数据记录、可视化与复现
- 9 第八篇：状态机与任务系统
- 10 第九篇：综合项目
- 11 第十篇：从 turtlesim 走向真实机器人

这样改完之后，教材入口会顺很多。学生不是一上来被命令砸中，而是经历一个非常自然的认知链条：

我会用 Linux 运行程序

→ 我知道机器人系统需要多个程序

→ 我发现多个程序需要通信

→ 我需要一个通信框架

→ ROS 出现

→ ROS1 需要 roscore

→ turtlesim 是第一个可观察的 ROS 世界

这就更接近你想要的那种“拆开、揉碎、一步步长出来”的写法。